AmnesicWallet — Technical documentation

Implementation specification, security model and declared limits

Document version: 2.2 — describes AmnesicWallet 2.2.0

Reference: `amnesicwallet.html` — SHA-256 hash published with every release, in `SHA256SUMS` and on the official website, amnesicwallet.com

---

Preliminary notice

This document describes how AmnesicWallet works technically. It is not a promotional document and it contains no security guarantees: the statements made here are verifiable by reading the source code, reproducing the build and running the tests described in section 8.

AmnesicWallet is free software distributed under the GNU General Public License v3 or later, **without any warranty**, express or implied. Custody of keys and backups is the sole responsibility of the user. The authors have no access to the wallets generated, keep no data and can under no circumstances recover seeds, passphrases or funds.

The software does not constitute financial, legal or tax advice. Users are responsible for verifying that their use complies with the regulations applicable in their jurisdiction.

---

1. Object and scope

AmnesicWallet is a web application contained in a single HTML file that generates cryptographic keys for cryptocurrency wallets according to public standards, and derives their public addresses.

1.1 Implemented functions

Function	Reference standard
Mnemonic phrase generation	BIP-39
Hierarchical deterministic derivation	BIP-32
Derivation paths	BIP-44, BIP-49, BIP-84, BIP-86, BIP-48
Taproot addresses	BIP-341
Multisig key ordering	BIP-67
Output descriptors and their checksum	BIP-380 and following
Interoperable threshold backup	SLIP-39
ed25519 curve derivation	SLIP-10
Ethereum address checksum	EIP-55
Coin identifiers	SLIP-44
Threshold backup over a BIP-39 seed (Shamir wallet)	No standard: scheme documented in 5.2
Binary representation of the backup	No standard: scheme documented in 5.4
Reading seeds made by Electrum (check only)	Electrum's own format, documented in 4.6

1.2 Deliberately unimplemented functions

- **Transaction building and signing.** Signing itself could be done offline, but spending needs a full wallet: coin selection, fee estimation, transaction formats, and eventually a connection to broadcast. Keeping all of that out keeps the program small enough to be read and checked. Users spend with a dedicated wallet, ideally a hardware one.
- **Balance lookups.** No queries to blockchains, explorers or third-party services.
- **Persistence.** No writes to `localStorage`, `sessionStorage`, `IndexedDB`, cookies or the filesystem. Cryptographic material exists only in the memory of the browser tab.
- **Telemetry.** No usage data collection, no identifiers, no network requests at runtime.

---

2. Architecture

2.1 Distribution model

The product is a single self-contained HTML file. All libraries are embedded into it at build time. At runtime no external resource is requested: no CDN, no remote fonts, no external images. The networks' symbols are drawn as SVG inside the file, so they appear even where no installed font contains them (as in Tails). QR codes are drawn locally as SVG `data:` images, without a canvas: Tor Browser refuses to read a canvas back, and would show a blank square.

Verifiable consequence: the file behaves identically on a device that has never had connectivity.

2.2 Source layout

File	Content
`src/core.js`	**All the cryptography**: entropy, BIP-39/32 derivation, addresses, descriptors, multisig, Shamir, SLIP-39, powers-of-2 grid. No DOM, no timers, no storage, no network.
`src/app.js`	The interface only: screens, texts, printing. It calls `core.js` for every cryptographic operation.
`src/index.html`	Structure and stylesheet.
`src/electrum-old-words.js`	The 1,626 words of Electrum 1.x seeds (MIT licence, from Electrum), used only to read them (4.6).
`src/crypto-shim.js`, `src/buffer-shim.js`	Adapters that let the `slip39` library run in a browser (2.5).

The separation means that the code tested by `npm test` (section 8.3) is the code that runs in the page: the test suite imports `core.js` itself, bundled with the same options.

2.3 Build chain

```
src/core.js + src/app.js + src/index.html
      │
      ├── esbuild — scripts/esbuild-options.js (shared with the tests)
      │     bundling, minification, ES2020 target
      │     alias:  crypto → src/crypto-shim.js
      │     inject: src/buffer-shim.js
      │
      └── scripts/build.js
            ├── inlines the bundle into index.html
            ├── refuses to write the file if the bundle contains a network API
            │   (fetch, XMLHttpRequest, WebSocket, EventSource, sendBeacon,
            │   RTCPeerConnection, importScripts), Math.random, a storage API
            │   (localStorage, sessionStorage, indexedDB, document.cookie),
            │   eval / new Function, or any URL other than the SVG namespace
            ├── adds the Content-Security-Policy (2.4) with the SHA-256 of the script
            ├── self-checks the result (one </script>, DOCTYPE first, hash matches)
            └── dist/amnesicwallet.html
```

Dependencies are pinned to exact versions in `package.json` and `package-lock.json`; `npm ci` installs exactly those, which makes the build reproducible byte for byte. `scripts/hash.js` writes the file's SHA-256 to `SHA256SUMS` and to the presentation page (`site/index.html`).

2.4 Content-Security-Policy

The published file contains, before any script:

```
default-src 'none'; script-src 'sha256-…'; style-src 'unsafe-inline';
img-src data:; base-uri 'none'; form-action 'none'
```

- `default-src 'none'` covers `connect-src`, `frame-src`, `font-src`, `worker-src` and every other fetch directive: the browser refuses any connection the page might attempt, whatever the code does.
- `script-src` allows only the one inline script whose SHA-256 is listed. Inline event handlers, injected scripts and a modified bundle would not run.
- `img-src data:` allows only the QR codes generated locally.

This is a second, independent barrier: the build already guarantees that no network code is present; the policy makes the browser enforce it even if that guarantee were wrong. Printed documents open in a blank window created by the page and inherit the same policy.

Limits, stated plainly: a policy delivered inside the page cannot forbid navigation (opening or redirecting a window to an address), and it does not cover WebRTC in every browser. The build's text checks, for their part, recognise the forbidden names only as written, not deliberately disguised code. Against those cases the protections are that the file contains no URL at all (checked at build time), that its code is public and reproducible, and — above all — running it on a disconnected device.

2.5 Libraries

Library	Version	Use
`@scure/bip39`	2.4.0	Mnemonic encoding and validation, PBKDF2 seed
`@scure/bip32`	2.4.0	Hierarchical deterministic derivation
`@scure/base`	2.4.0	bech32, bech32m, Base58, Base58Check
`@noble/hashes`	2.4.0	SHA-256, SHA-512, Keccak-256, RIPEMD-160, HMAC, PBKDF2
`@noble/curves`	2.4.0	secp256k1 (Taproot tweak, Ethereum/TRON public keys), ed25519 (Solana)
`slip39`	0.1.9	SLIP-39 splitting and reassembly
`qrcode`	1.5.4	QR codes, drawn locally

The `@noble` and `@scure` families are dependency-free implementations by the same author, independently audited and widely adopted. Version 1.1.0 removed four libraries used by earlier versions (`ethers`, `bech32`, `bs58`, `@noble/ed25519`), whose functions are now covered by the ones above: the file went from about 730 KB to about 360 KB.

The `slip39` library is written for Node.js and uses the `crypto` module and the `Buffer` global. Two adapters replace them:

- **`src/crypto-shim.js`** exposes only `randomBytes` (backed by `crypto.getRandomValues`), `pbkdf2Sync` and `createHmac` (backed by `@noble/hashes`).
- **`src/buffer-shim.js`** exposes `Buffer.from` and a few helpers, returning `Uint8Array`.

The adapters are exercised by the whole official SLIP-39 vector suite on every test run (section 8.3).

2.6 Language

The interface is in English only. There is no runtime translation layer: every string on screen is written directly in the source.

---

3. Entropy generation

3.1 Requirement

The security of a deterministic wallet depends entirely on the unpredictability of the initial value. A faulty generator makes every later operation irrelevant; real funds have been lost to generators that were far less random than they seemed.

3.2 Sources

System CSPRNG — always. `crypto.getRandomValues()`, fed by the operating system generator. 64 bytes are drawn for every key.

Typing dynamics. For every keystroke, the key value and a high-resolution timestamp (`performance.now()`) are recorded; the useful contribution lies mainly in the intervals. Thresholds: 20 keystrokes, 5 seconds, 10 distinct keys. On phones, whose on-screen keyboards do not report keys, the characters are read from the input field instead.

Pointer dynamics. Coordinates and timestamps of mouse or finger movement. Progress requires, jointly: 8 seconds, 100 events, 1000 px travelled and 8 changes of direction.

Coarse timers. Browsers that resist fingerprinting — Tor Browser, and so Tails — round `performance.now()` to tens of milliseconds. The intervals then carry less unpredictability; the thresholds are still reached in the same way, and the system CSPRNG, which is always part of the mix (3.3), does not depend on them.

Dice (optional). Rolls entered by the user: $\log_2 6 \approx 2.585$ bits each, so 50 rolls for 128 bits and 100 for 256 bits. With two dice thrown together, identical dice cannot be told apart and people tend to enter the smaller number first; a pair is then worth $\log_2 21 \approx 4.39$ bits, so the program asks for 30 double rolls (128 bits) or 59 (256 bits). The only source generated entirely outside the computer.

Each user source is reduced to a 32-byte SHA-256 digest; the raw samples are then cleared.

3.3 Combination

```
entropy = SHA-256( "AmnesicWallet/entropy/v1" ‖ CSPRNG(64)
                   ‖ 0x01 ‖ H(pointer) ‖ 0x02 ‖ H(dice) ‖ 0x03 ‖ H(keyboard) )[0 … n)
```

with `n` = 16, 20, 24, 28 or 32 bytes for 12 to 24 words. Each user source enters as a one-byte label and a fixed-length digest; a missing optional source is left out, label included. The input is therefore unambiguous: no two different sets of sources can produce it. The domain string makes the digest impossible to confuse with a hash computed for any other purpose.

Invariants, enforced in `combineEntropy` and covered by tests:

1. The CSPRNG contribution is **always present and never conditional**; the function has no code path without it.
2. The user sources are **added**, never substituted: they cannot disable or replace the CSPRNG.
3. Modelling SHA-256 as a random oracle, the output is unpredictable to anyone who cannot predict **all** the inputs. An attacker who knew or controlled the keyboard, the pointer and the dice would still face the 512 bits of the CSPRNG; a faulty CSPRNG would still be covered by the user sources.

Multisig with all keys generated locally. The first key uses the combination above; each further key calls `combineEntropy` again, with a **fresh 64-byte CSPRNG draw** and the same user digests. Keys are therefore independent as long as the CSPRNG is; if it returned the same bytes twice, the vault would contain a duplicate key, which `multisigAddress` refuses (4.4).

After use, the digests and the dice rolls are overwritten (best effort, see 6.3).

3.4 Checks on the generator

Before use, the raw CSPRNG output is subjected to three checks. The failure of any one of them **stops generation**, with no fallback.

Check	Failure condition
Constant value	All bytes identical
Repeatability	Two further 32-byte draws are identical
Balance (monobit)	Fraction of bits set to 1 outside [0.25, 0.75]

At start-up the presence and operation of the CSPRNG are verified; if the check fails, generation stays disabled.

The checks run on the generator's output, not on the final digest: SHA-256 output looks random even when its input is not, so a test there could only produce false alarms. (Version 1.0.x also tested the digest; that test was removed in 1.1.0 for this reason.)

Declared limit. These checks detect catastrophic failures only. No software check can measure the real entropy of a sequence: degraded but plausible-looking output would not be caught. That is what the independent sources are for.

---

4. Key derivation

4.1 BIP-39

The entropy is extended with a checksum equal to the first `ENT/32` bits of its SHA-256, split into groups of 11 bits, each indexing a word of the official English dictionary of 2048 words.

Words	Entropy	Checksum
12	128 bits	4 bits
15	160 bits	5 bits
18	192 bits	6 bits
21	224 bits	7 bits
24	256 bits	8 bits

The 64-byte seed is obtained with `PBKDF2-HMAC-SHA512`, 2048 iterations, salt `"mnemonic" || passphrase`, both NFKD-normalised. When a passphrase is created, the program refuses leading or trailing spaces, which are invisible on paper.

Only the English dictionary is used: localised dictionaries have uneven support in other wallets and would add a real risk of unrecoverability.

4.2 Derivation paths

Network	Path	Standard
Bitcoin — Native SegWit	`m/84'/0'/0'/0/i`	BIP-84
Bitcoin — Taproot	`m/86'/0'/0'/0/i`	BIP-86
Bitcoin — Nested SegWit (P2SH-P2WPKH)	`m/49'/0'/0'/0/i`	BIP-49
Bitcoin — Legacy	`m/44'/0'/0'/0/i`	BIP-44
Bitcoin — P2WSH multisig	`m/48'/0'/0'/2'`	BIP-48
Ethereum and EVM networks	`m/44'/60'/0'/0/0`	BIP-44
TRON	`m/44'/195'/0'/0/0`	SLIP-44
Solana	`m/44'/501'/0'/0'`	SLIP-10 (ed25519, hardened only)

These are the paths used when a wallet is **generated**; they do not change.

Checking a seed. The check shows the same paths for Account 1 and lets each network be checked separately on other accounts and derivations, because wallets do not all follow the same path. `n` is the account number minus one; every derivation that gives a different address has its own button:

Network	Derivations offered
Bitcoin	`m/84'/0'/n'/…`, `m/86'/0'/n'/…`, `m/49'/0'/n'/…`, `m/44'/0'/n'/…`, each with its receiving (`/0/i`) and change (`/1/i`) addresses; and Bitcoin addresses, in the four formats, on Ethereum's `m/44'/60'/0'/0/n` and TRON's `m/44'/195'/0'/0/n`
Ethereum	`m/44'/60'/0'/0/n`, `m/44'/60'/n'/0/0`, `m/44'/60'/0'/n`
TRON	`m/44'/195'/0'/0/n`, `m/44'/195'/n'/0/0`, and Ethereum's `m/44'/60'/0'/0/n`
Solana	`m/44'/501'/n'/0'`, `m/44'/501'/n'`, `m/44'/501'`, no path (the first 32 bytes of the seed as the ed25519 secret, as `solana-keygen` does by default), and `m/501'/n'/0/0` derived with BIP-32 on secp256k1, whose private key becomes the ed25519 secret (the old Sollet derivation)

Two derivations that give the same path for the account shown (for Account 1, `m/44'/60'/0'/0/0` twice) appear once.

Finding an address. Given an address of the seed being checked, the program recognises its network and format from its encoding and checksum, and recomputes addresses until it finds it: every derivation above for accounts 1–10 (Solana, Ethereum and TRON: up to 50 accounts or addresses per derivation), and for Bitcoin the first 50 receiving and change addresses of every account in all four purposes, plus the Ethereum and TRON paths, `m/0'/0/i`, `m/0'/1/i` and `m/0'/0'/i'` — about 4,250 addresses. An address outside this range is reported as not found.

4.3 Address construction

- **Legacy (P2PKH):** Base58Check(0x00 ‖ RIPEMD160(SHA256(pubkey)))
- **Nested SegWit (P2SH-P2WPKH):** redeem script `0x0014 ‖ hash160(pubkey)`, address Base58Check(0x05 ‖ hash160(redeem))
- **Native SegWit (P2WPKH):** bech32, witness v0, program `hash160(pubkey)`

- **Taproot (P2TR):** internal key `P` with even Y; tweak `t = H_TapTweak(x(P))`; output key `Q = P + t·G`; bech32m, witness v1, program `x(Q)`. No script tree (BIP-86).
- **Ethereum:** last 20 bytes of Keccak-256 of the uncompressed public key, with the EIP-55 mixed-case checksum.
- **TRON:** the same 20 bytes, Base58Check with version byte 0x41.
- **Solana:** SLIP-10 ed25519 private key at `m/44'/501'/0'/0'`, public key in Base58.

4.4 Multisig

Script `OP_m <pubkey…> OP_n OP_CHECKMULTISIG`, wrapped in P2WSH; public keys sorted according to BIP-67, so the address does not depend on the order in which keys were entered. From 2 to 15 keys; threshold from 1 to n (with a warning that a threshold of 1 lets any single key spend).

When the program creates every key of a vault, or computes it from seeds the user types, each key's public side — its `xpub` at `m/48'/0'/0'/2'`, the fingerprint of its seed and the path — can be shown and copied on its own, as co-signing programs ask for it when a vault is built key by key.

Accepted co-signer keys: an **xpub**, or the **Zpub** that Electrum shows for native-SegWit multisig (converted to the same key). Refused, each with its own explanation: private keys of any kind (xprv, yprv, zprv, Yprv, Zprv), testnet keys, keys labelled for another script type (ypub, zpub, Ypub), malformed keys, and **the same key twice** — a duplicate would let one person provide two signatures. Duplicates are recognised by what determines the derived keys (chain code and public key), not by the text: the same key given once as xpub and once as Zpub, or with altered metadata (depth, parent fingerprint, child number), is refused.

The descriptor produced is `wsh(sortedmulti(m, …))#checksum`, where every key generated in this program carries its origin, `[fingerprint/48h/0h/0h/2h]xpub…/0/*`, so Sparrow and hardware wallets recognise their own keys. Keys pasted from others appear without origin, since it cannot be known.

4.5 Watch-only descriptors

For a single-signature wallet, the account xpub and a descriptor are offered for monitoring without spending ability, e.g. `wpkh([fingerprint/84h/0h/0h]xpub…/0/*)#checksum` (respectively `tr(…)`, `sh(wpkh(…))`, `pkh(…)` for the other formats). The checksum follows BIP-380; the implementation is tested against the specification's example and against embit.

Check with a public key only. An account key shared by a wallet — `xpub`, `ypub` (Nested SegWit) or `zpub` (Native SegWit) — is read, re-labelled as a plain `xpub` and derived at `/0/i` (receiving) and `/1/i` (change), in any of the four Bitcoin formats. An `xpub` does not say which format it was used with, so the format is chosen on screen. A watch-only descriptor without key origin is produced. Private keys (`xprv`, `yprv`, `zprv`…), testnet keys and multisig keys (`Ypub`, `Zpub`) are refused, each with its reason.

4.6 Seeds made by Electrum (check only)

Electrum, from version 2.0, has a seed format of its own. Check wallet recognises it and reads it as Electrum does; the program never creates such seeds. Everything follows Electrum's `mnemonic.py`, `old_mnemonic.py` and `keystore.py`.

- **Normalisation** (`normalize_text`): NFKD, lower case, combining marks removed (those with a non-zero combining class, as Python's `unicodedata.combining`), single spaces, no space between two CJK characters. It applies to the words and to the passphrase, which Electrum calls the *seed extension*.

- **Type**: the hex form of HMAC-SHA512 with key `Seed version` over the normalised words starts with `01` (Standard), `100` (Segwit), `101` (2FA) or `102` (2FA Segwit). A `101` seed counts as 2FA only with 12 or at least 20 words, as in Electrum. Before that, 12 or 24 words all in the Electrum 1.x list are an Electrum 1.x seed.
- **Seed**: PBKDF2-HMAC-SHA512, 2,048 iterations, password the normalised words, salt `electrum` followed by the normalised passphrase.
- **Addresses**: Standard — BIP-32 from that seed, `m/0/i` (receiving) and `m/1/i` (change), P2PKH. Segwit — `m/0'/0/i` and `m/0'/1/i`, P2WPKH. The master public key is shown as Electrum shows it (an `xpub` at `m`, a `zpub` at `m/0'`), together with a descriptor with key origin.
- **Electrum 1.x**: the words give a hex seed (three words for every 32 bits, as `mn_decode`); the hex string is stretched with 100,000 rounds of SHA-256 into the private key; the master public key is that key's uncompressed point; address *n* of branch *c* is the uncompressed P2PKH address of `mpk + SHA256d("n:c:" ‖ mpk)·G`. These seeds have no passphrase.
- **2FA** seeds are recognised but not computed: their addresses are 2-of-3 multisig with keys of the TrustedCoin service.
- **Both at once.** About one BIP-39 seed in 256 also has the Standard prefix (one in 4,096 the Segwit one). Words that are valid both ways are shown as BIP-39, with a note and a button to read them as Electrum; Electrum's recent versions avoid making such seeds.
- **Finding an address** searches the first 200 receiving and 200 change addresses.

---

5. Backup splitting

The kind of wallet is chosen once, on the first screen of Generate wallet, before any randomness is collected:

Kind	What is created	Recovered with
Classic wallet	One BIP-39 phrase, 12–24 words	Any BIP-39 wallet
Shamir wallet	One BIP-39 seed, shown only as n parts of which m are needed (5.2)	This program; the reassembled seed works in any wallet
SLIP-39 wallet	n SLIP-39 sheets of which m are needed (5.3)	Trezor, Sparrow, Electrum, Keystone and others
Multisig vault	A Bitcoin m-of-n address (4.4)	Sparrow, Electrum or hardware wallets, with the descriptor

The choice changes only the interface. Classic and Shamir wallets draw their entropy and derive their addresses in exactly the same way; a Shamir wallet then passes the entropy to the functions of 5.2, unchanged. A Classic wallet can be divided afterwards with the sequential split of 5.1 ("Split into groups"); an existing BIP-39 seed can be given a threshold backup from Check wallet → Shamir backup.

5.1 Sequential split

Partitioning of the word sequence into consecutive groups, offered on a Classic wallet. No cryptographic transformation; reassembly is manual and needs no software.

Property: knowing some groups reduces the search space by the words they contain. The program states, for the chosen split, how many words someone holding every part but one would still be missing, and what that means: with 12 words in 3 parts, 4 missing words (40 bits after the checksum) are within reach of a single computer; with 24 words in 3 parts, 8 missing words are not. No verification code is attached to these parts: the words are numbered and carry the BIP-39 checksum, and a code would only help someone guessing a missing part.

5.2 Threshold scheme for an existing seed (Shamir)

Shamir Secret Sharing applied byte by byte to the BIP-39 entropy. It is used when a Shamir wallet is created, and from Check wallet on any existing BIP-39 seed, including ones created elsewhere, which SLIP-39 cannot represent.

- Finite field GF(2^8), irreducible polynomial `0x11b`; exponential and logarithm tables built with **generator 3**. (Generator 2 is not primitive for `0x11b`: it only reaches 51 of the 255 non-zero elements.)
- For each byte, a polynomial of degree m − 1 whose constant term is the secret byte and whose other coefficients are drawn from the CSPRNG, **uniformly, zero included**. Uniform coefficients are what the proof of perfect secrecy requires; excluding zero (as some implementations do) would slightly bias the parts.
- Parts are the values at x = 1 … n (n ≤ 16, threshold 2 ≤ m ≤ n), each encoded as a BIP-39 mnemonic of the same length as the seed.
- Reconstruction by Lagrange interpolation at x = 0. The part number, the length of every part and the uniqueness of the numbers are checked.
- Verification code: the first 2 bytes of SHA-256 of the entropy, in hexadecimal (4 characters).

Security property. With fewer parts than the threshold, every value of the secret remains exactly equally likely: this is information-theoretic secrecy, not computational hardness. The verification code, printed on every sheet, is the only information that is not perfectly hidden: it reveals 16 bits of a hash, leaving at least 2^{112} candidates for a 12-word seed.

Detection of errors. With fewer parts than the threshold, or a wrong part, interpolation still returns a value — which is always a valid-looking seed. Only the verification code detects it (a wrong result passes with probability 1/65,536). Without the code the program says plainly that the result cannot be confirmed.

What is printed. Each sheet carries the words and, in a corner with no label, the part number and the verification code ("2 · A3F9"): the number is needed to recombine, the code to confirm the result. Nothing else — neither the program's name, nor the threshold, nor how many parts exist. Sheets printed by 1.1–1.3 also showed "Part 2 of 5", the threshold and a line naming the program; those printed by 1.0.x showed only "Part 2", and the code had to be copied by hand.

Compatibility. The format is frozen. `tests/vectors/shamir-compat.json` contains parts produced by version 1.0.1; every combination of three of them must reassemble, and the test suite fails otherwise.

Declared limit. This scheme is not a public standard; reassembly needs this program or a reimplementation of this section. The file should be kept together with the parts.

5.3 SLIP-39

Implementation of SLIP-39 through the `slip39` library and the adapters of 2.5.

- One group, threshold m of n sheets (2 ≤ m ≤ n ≤ 7 in the interface); 128-bit master secret, i.e. 20-word sheets.
- Shares are created with the **extendable-backup flag** set (ext = 1), as the current revision of the specification recommends and as Trezor does. Programs that predate that revision may not read them correctly.
- Iteration exponent 1 (20,000 PBKDF2-SHA256 iterations in the encryption of the master secret), the default of Trezor's reference implementation. Version 1.0.x used exponent 0; the exponent is written in the sheets, so older sheets remain readable.
- The passphrase may contain only printable ASCII, as the specification requires; the rule is enforced both when the passphrase is chosen and at recovery.
- A printed sheet carries only its words: each share already contains, inside them, its index, the threshold and the identifier shared by the set.
- **Self-check:** before the sheets are shown, every subset of m sheets is recombined and must return the master secret (at most 35 recombinations, about one second on a desktop computer).
- Recovery shows Bitcoin in all four formats, because the owner of a Trezor backup may use any of them.

Difference from BIP-39. The master secret is used **directly** as the BIP-32 seed, without PBKDF2. For the same entropy, BIP-39 and SLIP-39 give different wallets; their shares are not interchangeable.

Passphrase behaviour. A wrong passphrase gives no error: it decrypts to another master secret, i.e. a different, empty wallet.

Sheets produced by the program have been recovered with Trezor's own reference implementation (`python-shamir-mnemonic`), for every subset of three out of five, with matching addresses (section 8.5).

5.4 Binary representation (powers-of-2 backup)

Every BIP-39 word is identified by its number in the dictionary, written as a sum of powers of two across twelve columns (2048 … 1).

Numbering from 1. The internal BIP-39 index starts at 0; the printed grid uses 1–2048. With zero-based numbering the first word (`abandon`) would have no marks at all and look like an unfilled row. With one-based numbering no row is ever empty.

The grid shows neither words nor numbers. Reading it back needs the numbered list of BIP-39 words, which the program can also print.

The same grid is offered for each Shamir part (a part is itself a BIP-39 phrase; its number and the verification code go in a corner under the grid, as on its printed sheet; a Classic group keeps the positions of its words in the seed) for each SLIP-39 sheet, and for each key of a multisig vault. SLIP-39 words come from their own list of 1024 words, numbered 1–1024 in the order of the official `wordlist.txt` , so a SLIP-39 grid has eleven columns (1024 … 1) and is read back with the numbered SLIP-39 list, which the program prints too. The grids of all the parts, all the sheets or all the keys can also be printed together, one per page.

---

6. Security model

6.1 Assumptions

- The device on which the software runs is not compromised.
- The file executed matches the published one, verifiable through its SHA-256.
- The browser, its JavaScript engine and its CSPRNG behave according to their specifications.

6.2 Threats addressed

Threat	Countermeasure
Network exfiltration	No network code (checked at build time) and a Content-Security-Policy that makes the browser refuse any connection
Injected or modified script	CSP allows only the script with the published hash
Persistence to disk	No storage APIs (checked at build time)
Faulty or manipulated CSPRNG	Mixing with independent sources, including one outside the computer (dice)
Catastrophic generator failure	Blocking checks on the raw output
Discovery of the paper backup	Passphrase; threshold splitting; multisig
Partial loss of the backup	Threshold schemes (Shamir, SLIP-39); multisig
Transcription error	BIP-39 checksum, verification code, "check again" function
Mixing up co-signer keys	Refusal of private keys, duplicates, testnet and wrong script types; key origins in the descriptor

6.3 Threats not addressed

The model does **not** protect against:

- a compromised operating system, keyloggers, malware reading process memory;
- physical compromise of the device during or after generation;
- observation of the screen (video recording, reflections, bystanders);
- user error in keeping the backups, or loss of the passphrase;
- vulnerabilities in the browser engine or in the embedded libraries;
- the clipboard: words copied with the Copy button stay there until overwritten, and some systems keep a clipboard history;
- **memory remanence.** JavaScript offers no way to guarantee that a value is erased: strings are immutable and the garbage collector decides when memory is reused. The program overwrites the buffers it controls, and "Generate a new wallet" drops every secret of the session (wallet, seeds being checked, multisig keys) and removes the hidden frame used for

printing (pages are printed from a frame of the page itself, never from a new tab); but only closing the tab — better, shutting down a live system such as Tails — releases everything.

For these reasons the documentation recommends running the tool on a system isolated from the network, preferably booted from removable media without persistence.

---

7. Personal data

The software does not collect, process or transmit personal data. There are no servers, endpoints or recipients. All material lives in the memory of the browser tab for the duration of the session; no cookies or storage entries are created, and no file — except when the user presses *Save as .txt* next to a new wallet's words, a Shamir part, a SLIP-39 sheet or a multisig key (one, or all of them in a single file): those numbered words (with a part's number and verification code; all the keys of a vault end with its descriptor) are then handed to the browser as a download (a `data:` link built in the page), saved wherever the browser saves downloads.

Printed documents are produced locally. The Seed Card and the address lists carry neutral titles and no program name, to limit what an accidental discovery reveals. Shamir parts and SLIP-39 sheets carry only what is needed to use them years later: their number, the total, the threshold and — for Shamir — the verification code.

---

8. Verifiability

Every statement in this document can be verified independently.

8.1 Integrity of the distributed file

```
sha256sum amnesicwallet.html                       # Linux, macOS
Get-FileHash amnesicwallet.html -Algorithm SHA256  # Windows
```

The value must match `SHA256SUMS`, the release notes and the official website, `amnesicwallet.com`. See `docs/VERIFICATION.md`.

8.2 Build reproducibility

```
git clone https://github.com/MrAmnesic/amnesicwallet.git
cd amnesicwallet
npm ci
npm run build
sha256sum dist/amnesicwallet.html
```

Continuous integration performs exactly this on every change (with dependency install scripts disabled) and compares the result with the committed file, `SHA256SUMS` and the hash on the presentation page. A pull request that does not match fails; a change pushed directly to `main` without the rebuilt file receives it in a separate commit made by CI, from the output of the checks — so the committed file is always one that CI built from the public sources. The release stops if the

tagged file differs from a fresh build, and the website is published from the committed file after the same comparison (`scripts/prepare-site.js`).

8.3 Test suite

```
npm test
```

`scripts/test.js` bundles `tests/core.test.js` together with `src/core.js` , using the build's own esbuild options and adapters, and runs it. About 3,500 checks, all with expected values from outside the project:

Area	Source of the expected values
BIP-39 entropy ↔ words ↔ seed ↔ root xprv	24 official vectors (trezor/python-mnemonic)
Bitcoin addresses	Published examples of BIP-44, BIP-49, BIP-84, BIP-86
Addresses of every format and network, account xpubs, fingerprints, BIP-48 xpubs, Zpub handling	Computed with **bip_utils** and **embit** (Python) for 5 mnemonics × 2 passphrases
Multisig vaults (addresses at indexes 0 and 5, key order, descriptor checksum) and every refusal	embit; constructed invalid keys
Every derivation offered by the check, for accounts 1, 2 and 5; change branches; the address search; addresses and descriptors from account xpubs, ypubs and zpubs, and every refusal	Computed with **bip_utils**, **embit** and PyNaCl for 3 mnemonics (`tests/vectors/paths.py`); Bitcoin cross-checked between the two libraries
Electrum seeds: types, BIP-32 seeds (including Japanese, Chinese and Spanish words and Unicode passphrases), master public keys, receiving and change addresses of Standard, Segwit and 1.x seeds, the address search	Electrum's own test values (`tests/test_mnemonic.py` , `tests/test_wallet_vertical.py`); more seeds and addresses computed with Python's `hashlib` and **bip_utils**, cross-checked with bip_utils' own Electrum module (`tests/vectors/electrum.py`)
Diagnosis of a mistyped seed (position, suggestions, checksum)	Official BIP-39 word list and vectors; constructed mistakes
SLIP-39 word numbers for the powers-of-2 grid	Official `wordlist.txt` of trezor/python-shamir-mnemonic
Ethereum checksum	EIP-55 examples
Solana derivation	SLIP-10 ed25519 official vectors
Descriptor checksum	BIP-380 example
SLIP-39 recovery, including the xprv	45 official vectors (trezor/python-shamir-mnemonic), 15 valid and 30 that must be rejected
SLIP-39 creation	Round trips for several thresholds, with and without passphrase; fewer sheets than the threshold must fail
GF(2^8)	FIPS-197 examples; inverse and commutativity over the whole field
Shamir split and reassembly	All subsets of every size, 5 lengths × 5 configurations; below-threshold subsets must not give the secret
Shamir compatibility	Parts produced by version 1.0.1
Entropy	The formula of 3.3 recomputed; every source changes the result; invalid lengths and sources refused; broken CSPRNG output stops generation
Collectors, dice, grid, sequential split	Simulated clock; exact counts

Canonical mnemonic `abandon × 11 + about` , no passphrase:

Network	Expected address
Bitcoin Native SegWit	`bc1qcr8te4kr609gcawutmrza0j4xv80jy8z306fyu`
Bitcoin Taproot	`bc1p5cyxnuxmeuwuvkwfem96lqzszd02n6xdcjrs20cac6yqjjwudpxqkedrcr`
Bitcoin Nested SegWit	`37VucYSaXLCAsxYyAPfbSi9eh4iEcbShgf`
Bitcoin Legacy	`1LqBGSKuX5yYUonjxT5qGfpUsXKYYWeabA`
Ethereum	`0x9858EfFD232B4033E47d90003D41EC34EcaEda94`
TRON	`TUEZSdKsoDHQMeZwihtdoBiN46zxhGWYdH`
Solana	`HAgk14JpMQLgt6rVgv7cBQFJWFto5Dqxi472uT3DKpqk`

`npm run test:ui` then uses the built file itself, in the three browser engines (Chromium, Firefox, WebKit), on a computer screen and on two phone sizes, and once more in Firefox with Tor Browser's settings at its *Safer* level (fingerprinting resistance with coarse timers, no WebGL, no WebRTC, no JIT, no WebAssembly, no MathML), on a computer screen and the narrowest window. It creates a wallet from start to finish and checks that the words form a valid seed and that the addresses shown are the ones that seed gives; it checks known seeds in Check wallet against the independent values above, together with a mistyped word, each network's accounts and derivations, change addresses, the address search, an Electrum seed, and the check with a public key only; and it fails on any page error, any network request, or any screen wider than the display. Canvas read-back is refused throughout, as Tor Browser does. Every character the page displays has been checked against the fonts Tor Browser ships for Linux, the only ones it uses. Continuous integration runs it on every change.

8.4 Verifying the absence of network traffic

Run the file with a traffic analyser active, or on a physically disconnected device: no outbound packet should be observed. In the browser's developer tools, any attempted connection would appear as a Content-Security-Policy violation.

8.5 Cross-checking

Import the same words into an independent implementation — Sparrow or Electrum, still offline — and compare the derived addresses. Agreement between independent implementations is the most significant verification available to an end user.

For release 1.1.0, beyond the test suite, the maintainers drove the built page in a browser through every function (generation with each source, 24 words with a Unicode passphrase, dice, Shamir split and recovery, SLIP-39 creation and recovery, multisig in both modes, every check path, printing), and compared what the page displayed with bip_utils, embit and python-shamir-mnemonic. They also checked 300 further seeds (5 lengths, 3 passphrases including Unicode) — 12,481 values — and found them identical to those of version 1.0.1.

8.6 Code inspection

All cryptography is in `src/core.js`, about 650 lines. Elements verifiable by direct inspection:

- the entropy combination (`combineEntropy`) and its unconditional CSPRNG draw;
- the Shamir field, split and reassembly (`GF`, `shamirSplit`, `shamirCombine`);
- the derivation paths declared in plain text for each network;

- the absence of `Math.random`, network and storage APIs — also enforced by `scripts/build.js` on the final bundle.

---

9. Declared limits

No independent audit. The code has not undergone a paid professional security review. Passing official vectors attests to the correctness of the derivations, not to the absence of vulnerabilities.

No long operational track record. The project does not have the years of use on significant value that characterise established implementations. That experience cannot be replaced by design arguments.

Limited reach of the generator checks. See 3.4.

Dependence on the program for Shamir parts. See 5.2. SLIP-39 does not have this limitation.

Dependence on the execution environment. See 6.3.

Electrum 2FA seeds. They are recognised, not computed (4.6).

Reach of the address search. The search in Check wallet covers the derivations and ranges listed in 4.2. An address further along (for example the 60th address of an account, or account 11) is not found by the search, but can still be reached with the account buttons and the list of addresses.

For significant amounts, hardware devices and multisig configurations with keys generated by different tools are the stronger choice.

---

10. Licence and references

The software is distributed under the GNU General Public License, version 3 or (at your option) any later version; the full text is in `LICENSE`. Anyone who distributes a modified version must release its source code under the same licence and state the changes made. The bundled libraries (@noble, @scure, qrcode, slip39) and the Electrum 1.x word list taken from Electrum are MIT licensed, which is compatible with the GPL.

Reference specifications

- BIP-32 — Hierarchical Deterministic Wallets
- BIP-39 — Mnemonic code for generating deterministic keys
- BIP-43 / BIP-44 / BIP-49 / BIP-84 / BIP-86 — Purpose field and derivation schemes
- BIP-48 — Multi-Script Hierarchy for Multi-Sig Wallets
- BIP-67 — Deterministic Pay-to-script-hash multi-signature addresses
- BIP-341 — Taproot: SegWit version 1 spending rules
- BIP-380 — Output Script Descriptors
- EIP-55 — Mixed-case checksum address encoding
- SLIP-10 — Universal private key derivation from master private key
- SLIP-39 — Shamir's Secret-Sharing for Mnemonic Codes
- SLIP-44 — Registered coin types for BIP-0044

- FIPS-197 — Advanced Encryption Standard (arithmetic in GF(2^8))
- Electrum — seed version system and key derivation (`electrum/mnemonic.py`, `electrum/keystore.py`)

Security reports. Vulnerabilities should be reported following `SECURITY.md`, privately and before any public disclosure.